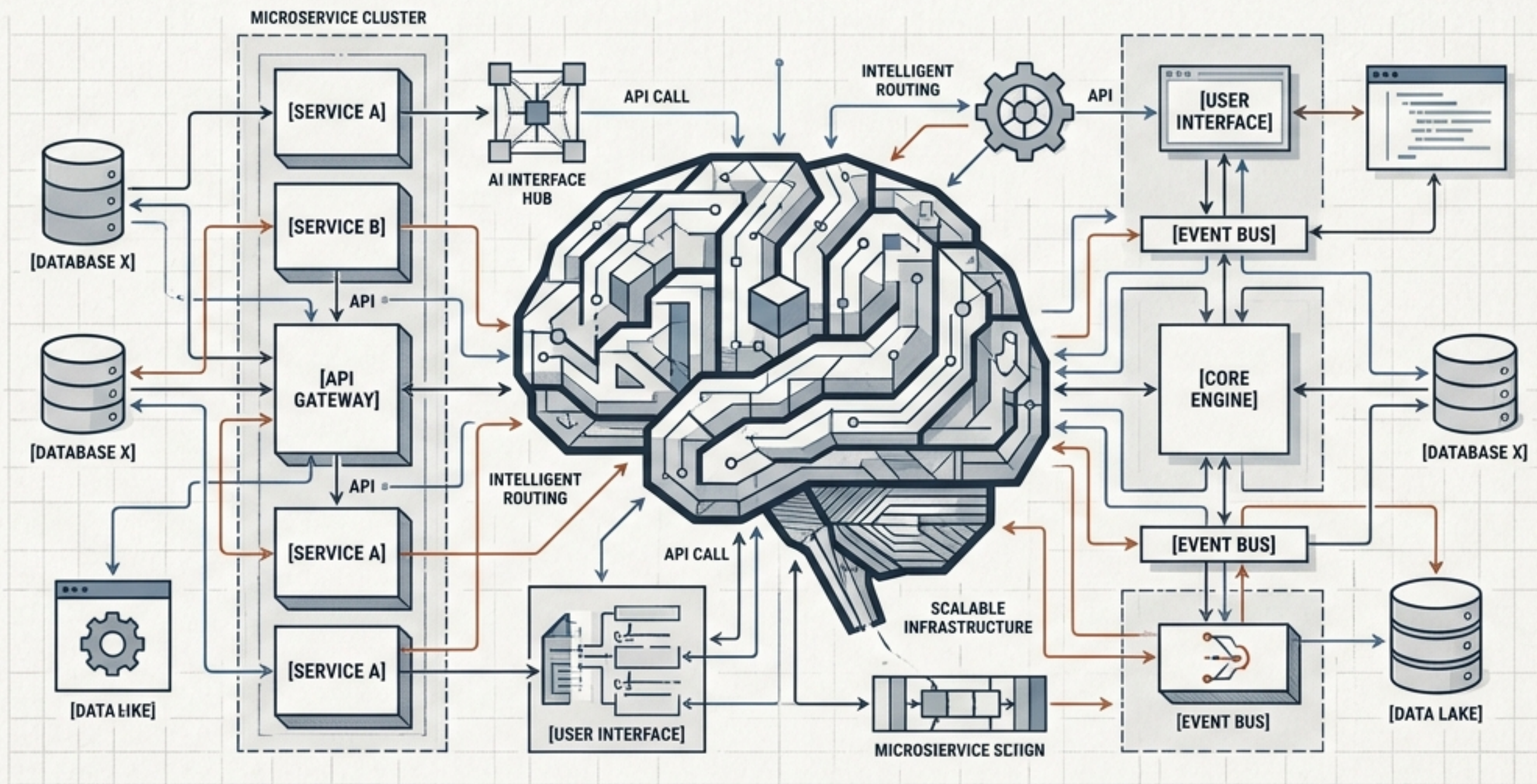
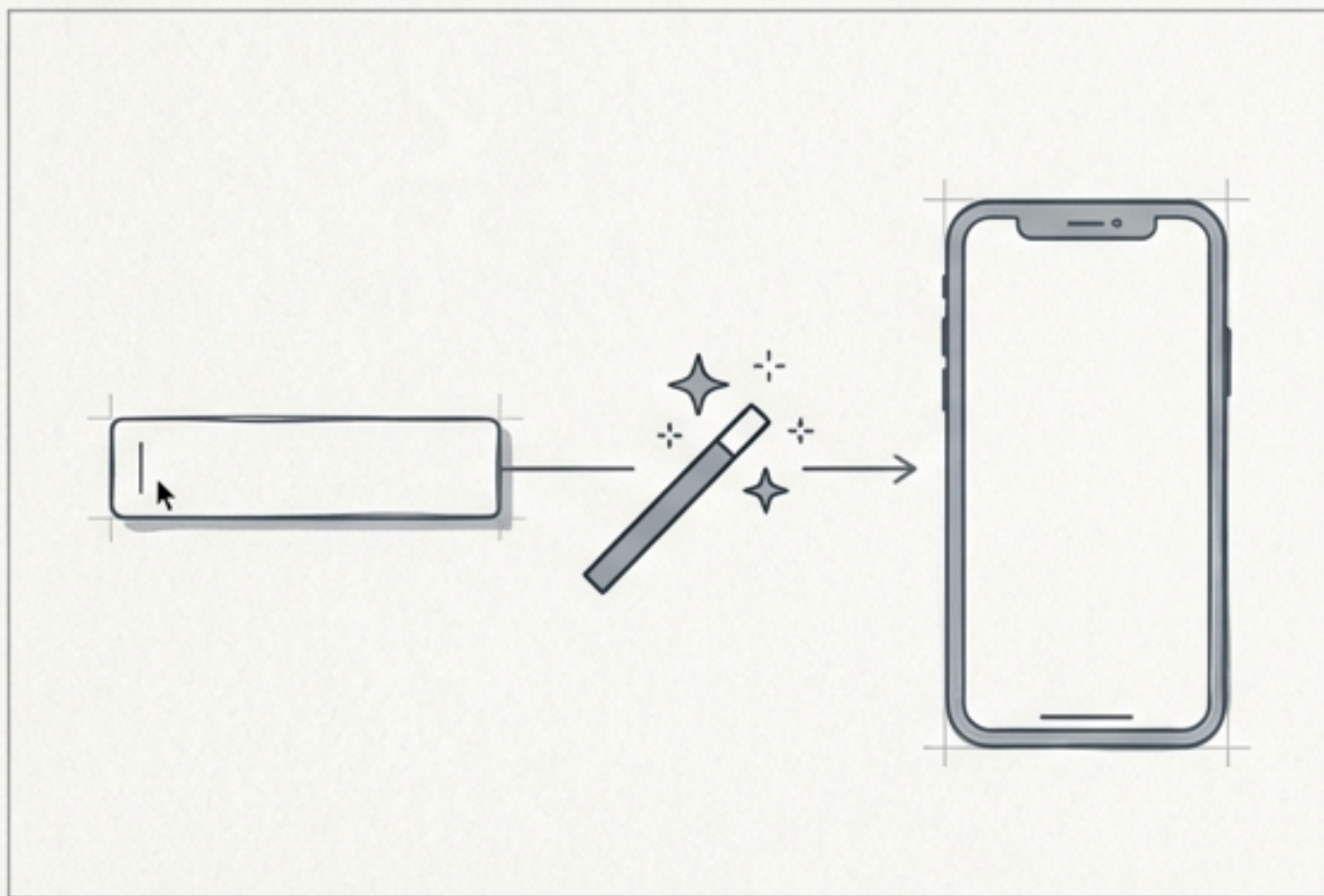




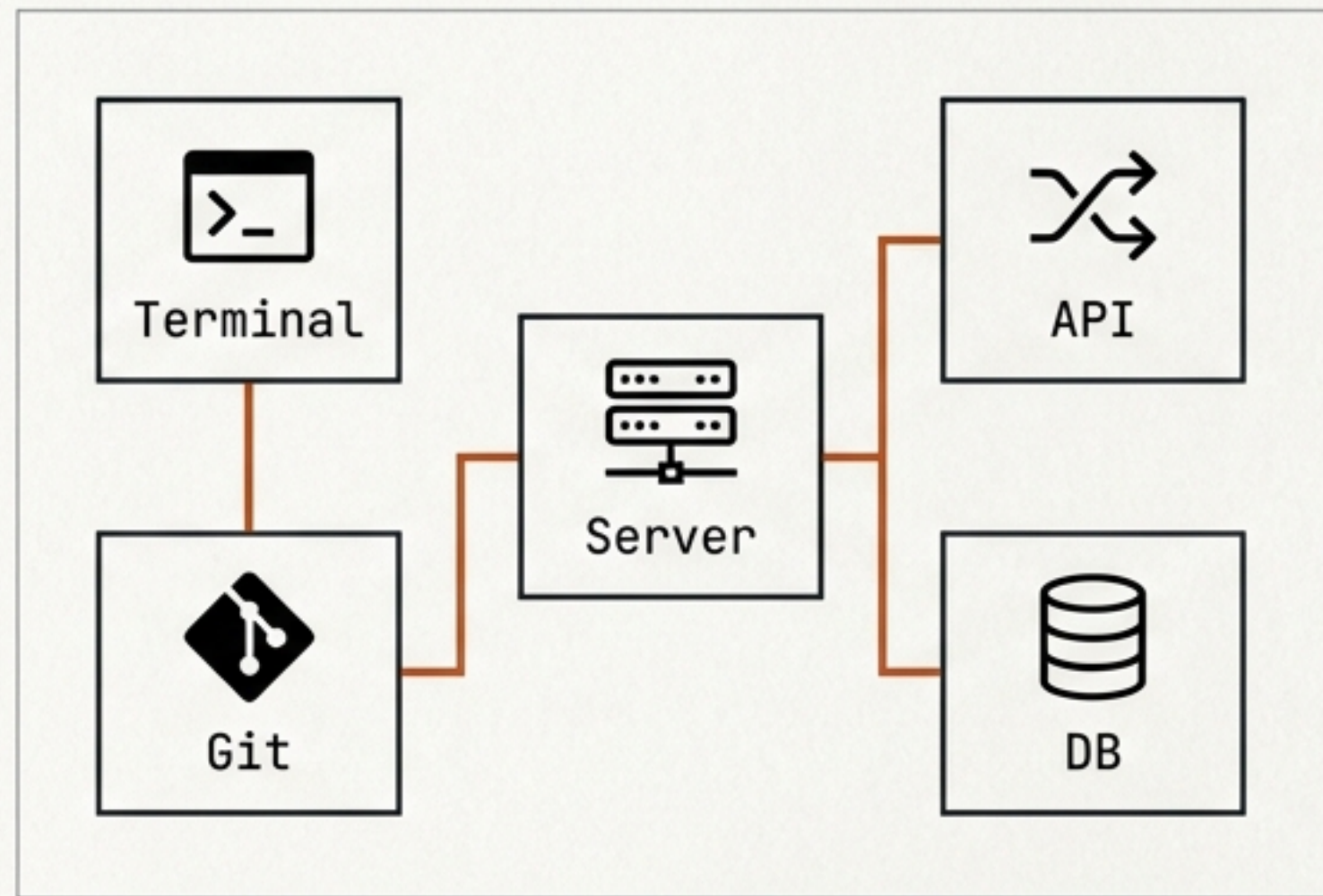
바이브 코딩 (Vibe Coding) 아키텍처 가이드

코딩은 AI가 합니다. 당신은 전체 '그림'만 이해하세요.

프롬프트 한 줄로 앱이 똑딱 만들어지는 마법은 없습니다



기대: 프롬프트 입력 → 완성된 앱



현실: AI는 코드만 짤 뿐, 그 코드를 조립하고
배포하는 환경은 당신이 통제해야 합니다.
당신은 이제 코더가 아니라 '**시스템 아키텍트**'입니다.

AI 개발 환경을 구성하는 3대 핵심 도구



대형 언어 모델 (두뇌)

ChatGPT, Gemini, Claude.
개발의 논리와 코드를
생성하는 지능.



커맨드 라인 인터페이스 (명령어)

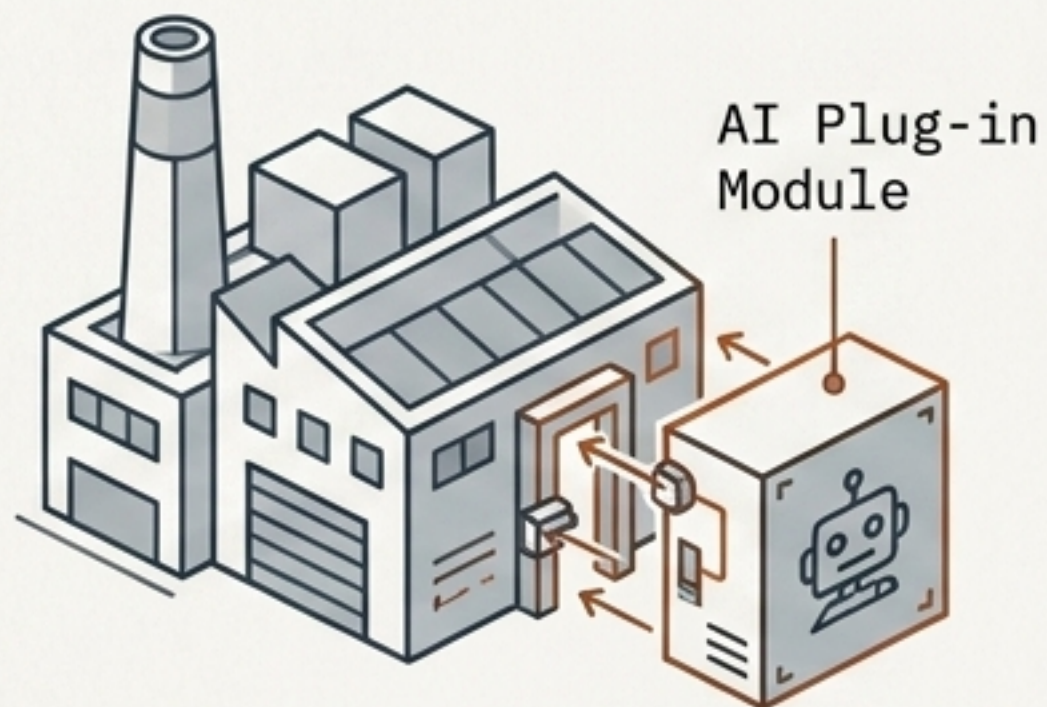
CMD 창처럼 텍스트 한 줄로
컴퓨터에 직접 명령을
내리는 창구.



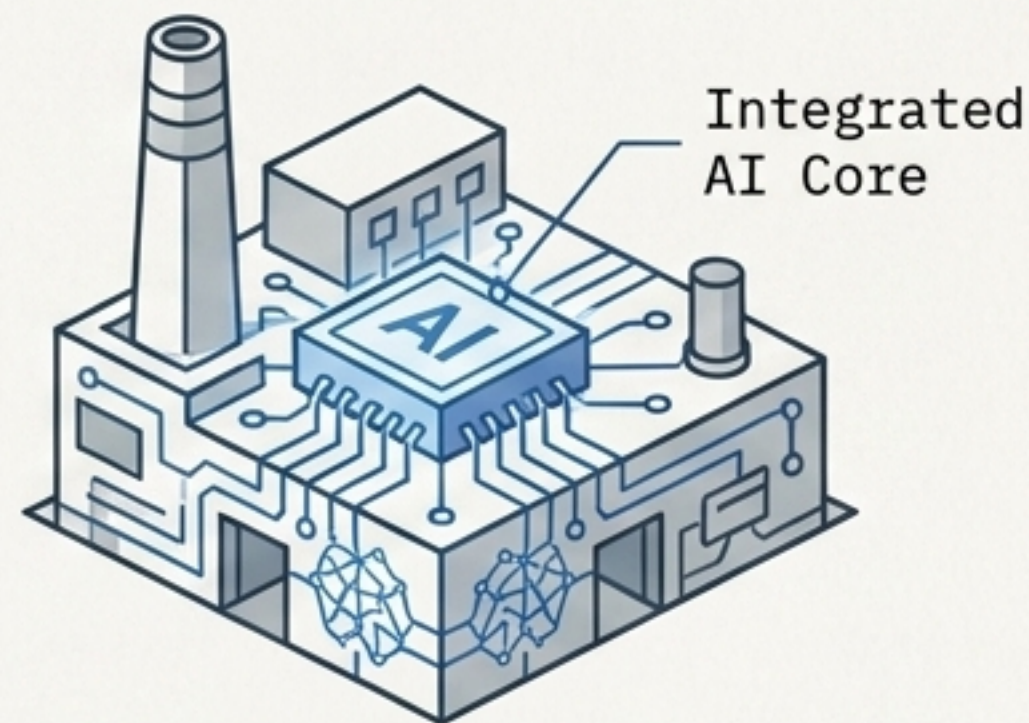
통합 개발 환경 (공장)

코드 편집, 컴파일, 빌드,
실행을 한 곳에서 처리하는
소프트웨어 공장.

어떤 공장을 선택할 것인가? (VS Code vs. Cursor)



- 방식: 표준 개발 환경 + AI 모듈 확장
- 특징: 전 세계 가장 표준적인 IDE.
- AI 연동: Claude 등의 플러그인(익스텐션)을 별도로 설치하여 우측 아이콘을 통해 연동.



- 방식: AI 내장형(Native) 개발 환경
- 특징: 다운로드 후 설치하면 내부에 AI가 완전히 통합되어 있음.
- AI 연동: 우측 채팅창이 기본 탑재. 질문 즉시 코드를 생성하고 기존 소스를 바로 수정 가능. 초보자에게 직관적.

AI와의 소통 방식 진화 (프롬프트에서 하네스까지)



1단계: 프롬프트 (Prompt)

질문의 체계화. 초기의 방식.
어떻게 명확하게 질문을 입력할
것인가에 집중.



2단계: 컨텍스트 (Context)

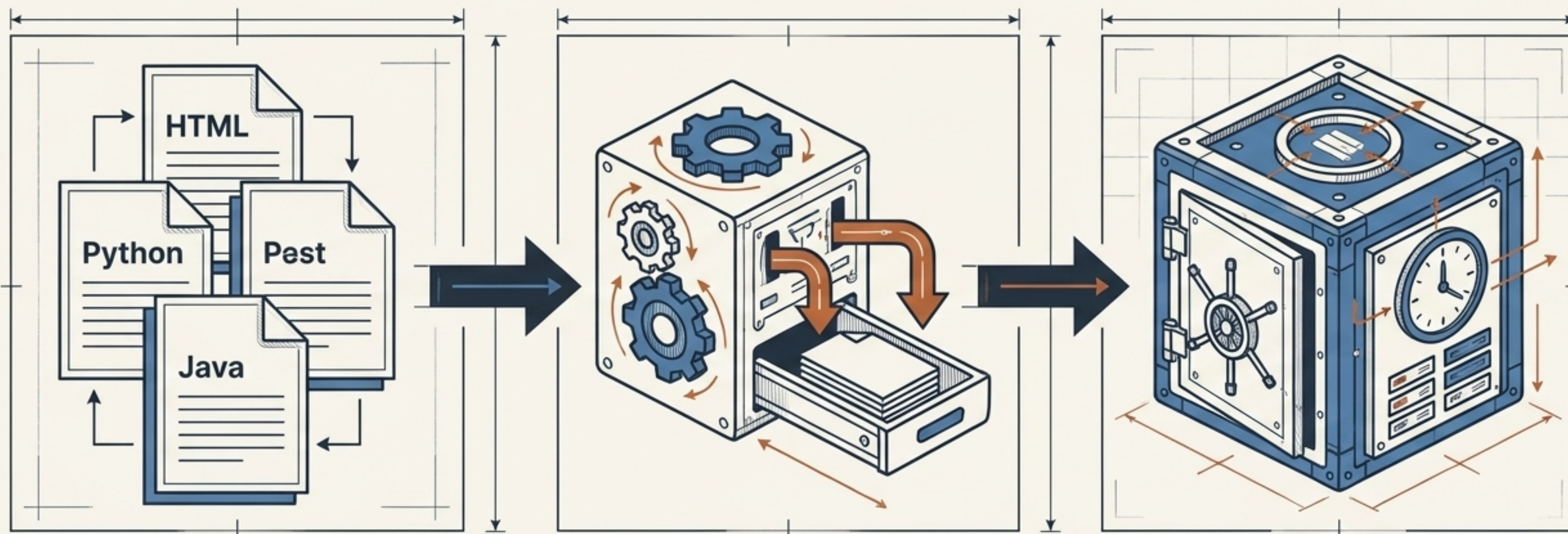
전후 맥락의 전달. 현재의 방식.
AI가 스스로 상황을 판단할 수 있도록
전체 코드와 환경 정보를 통째로 제공.



3단계: 하네스 & 에이전트 (Harness & Agent)

자율 대리인. 미래의 방식.
단순 코딩을 넘어 기존 코드 테스트,
안정성 검토까지 주도적으로 수행.

소스 코드의 탄생과 깃허브(GitHub)의 역할

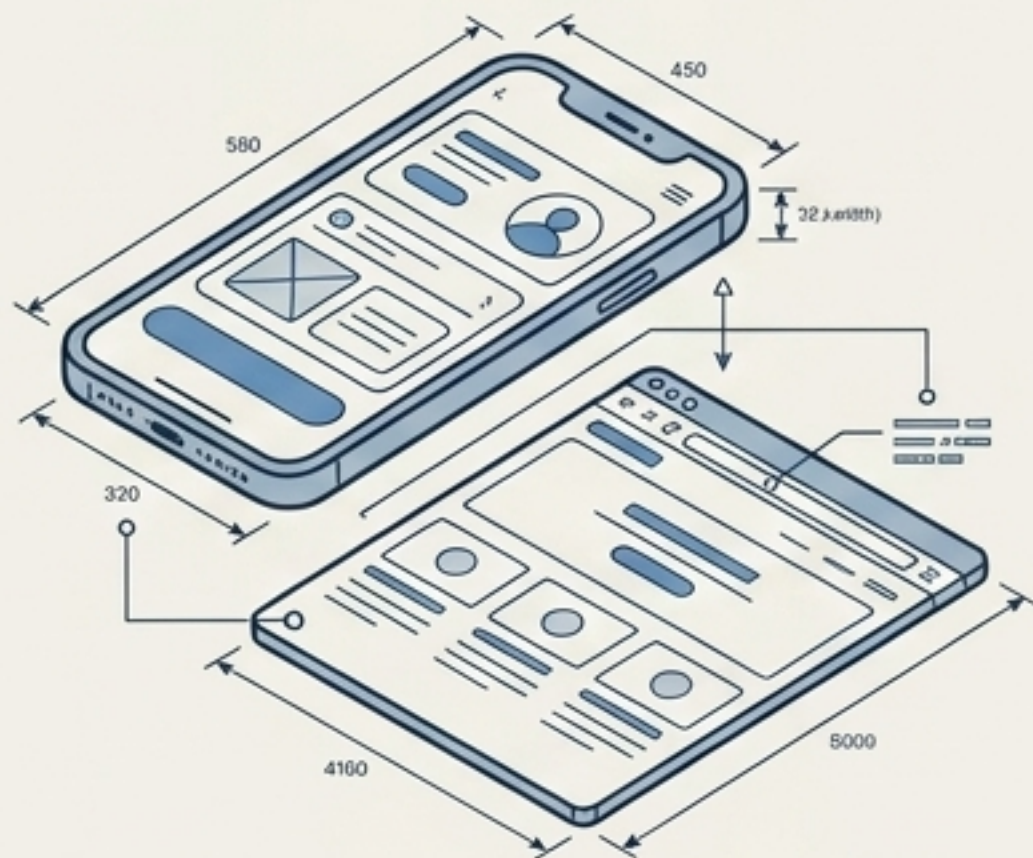


소스 (Source): AI가 작성한
원시 텍스트.

컴파일 & 빌드: 텍스트 형태의
소스를 실행 가능한 '한 덩어리'의
환경으로 묶고 변환하는 과정.

저장과 협업의 타임머신: 프로젝트
별로 생성/수정 내역을 관리하고
안전하게 올려두는 원격 저장소.

소프트웨어의 두 세계: 프론트엔드와 백엔드



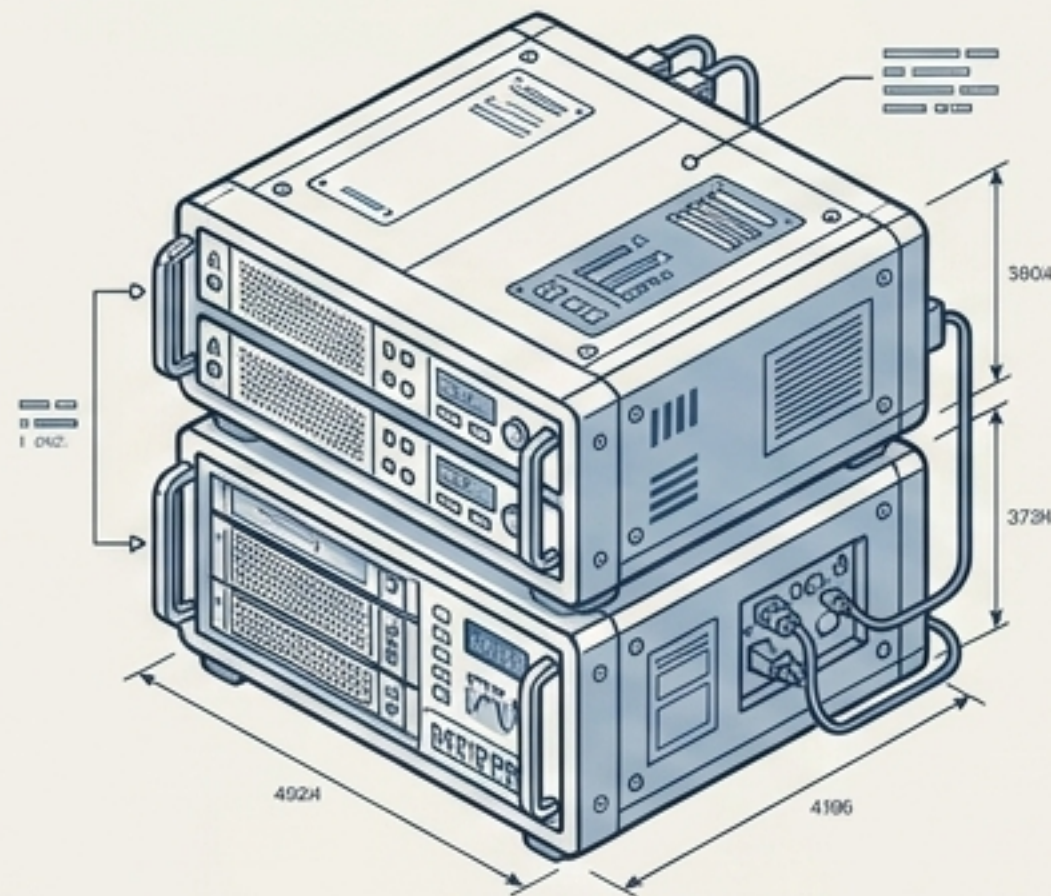
프론트엔드 (화면)

역할: 사용자의 눈에 보이고 직접 만지는 영역.

구동 위치: 스마트폰 단말기, 브라우저.

언어/도구: [Web] HTML, CSS, JavaScript /
[App] React, Flutter, React Native.

결과물: 앱스토어 등록 또는 웹서버 배포.



백엔드 (서버)

역할: 보이지 않는 곳에서 데이터를 처리하는 두뇌.

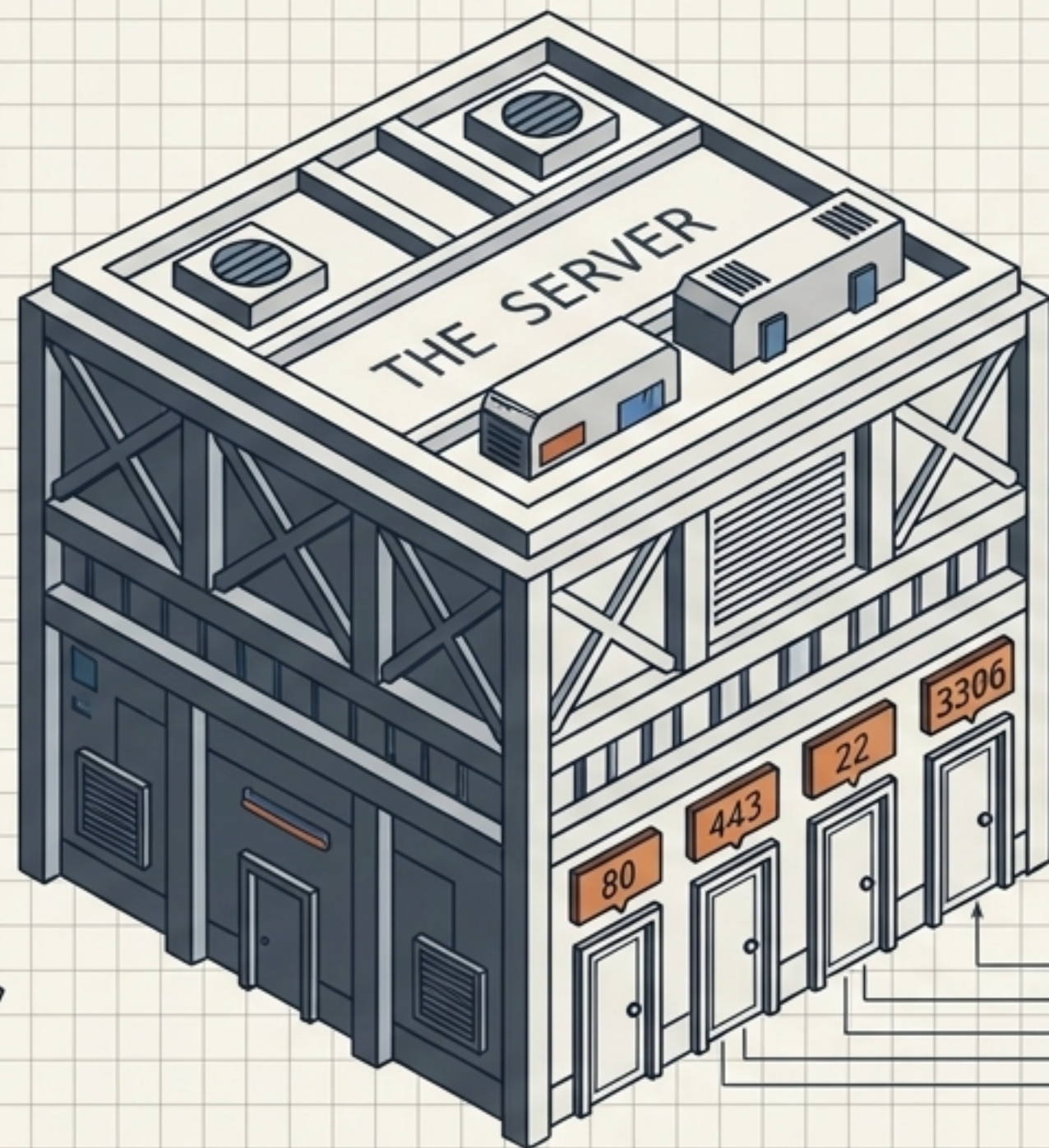
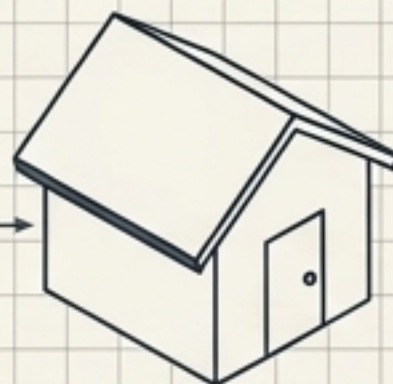
구동 위치: 데이터센터의 서버 컴퓨터.

언어/도구: Spring Boot (Java), Node.js (JS),
FastAPI (Python).

서버를 띄운다는 것, 그리고 포트(Port)의 비밀

Localhost (내 집):

다른 곳이 아닌 '내 컴퓨터 자체'를 가리키는 고유 주소.

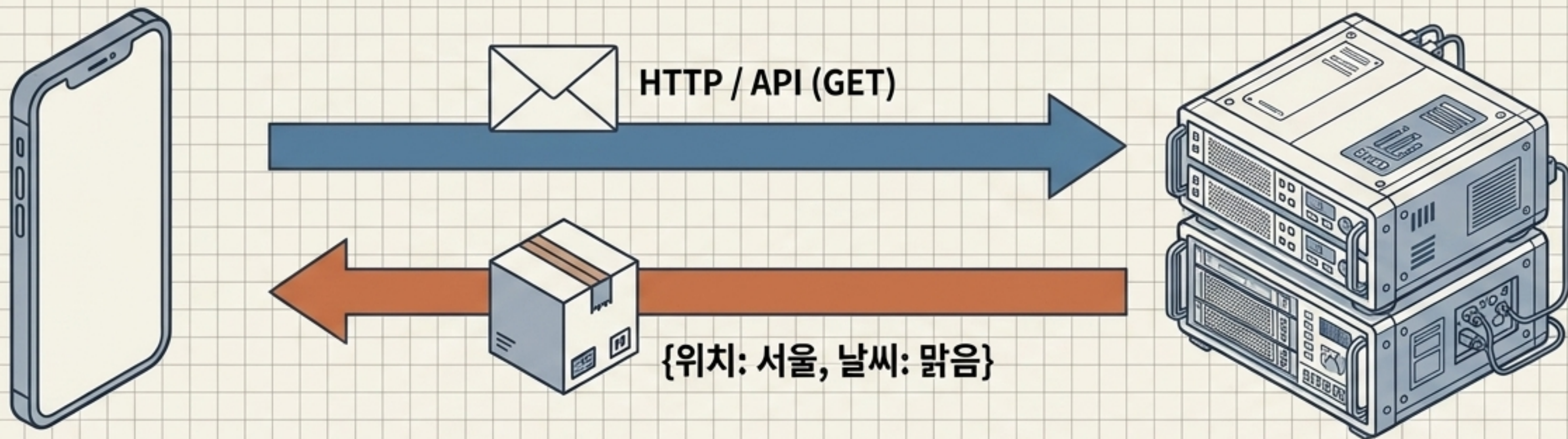


Ports (지정된 출입구):

서버 프로그램이 실행될 때 반드시 가지는 고유 번호(문). 데이터가 정확한 프로그램을 찾아가기 위한 주소입니다.

- 80: Web Server (웹 페이지 전송)
- 443: HTTPS (보안 통신)
- 22: SSH (원격 접속)
- 3306: Database (데이터 저장소)

프론트와 백엔드의 대화 규칙 (HTTP, API, JSON)



HTTP

대화의 프로토콜 (약속).
HTML 웹페이지나 데이터를 주고받기
위한 범용적인 통신 규칙.

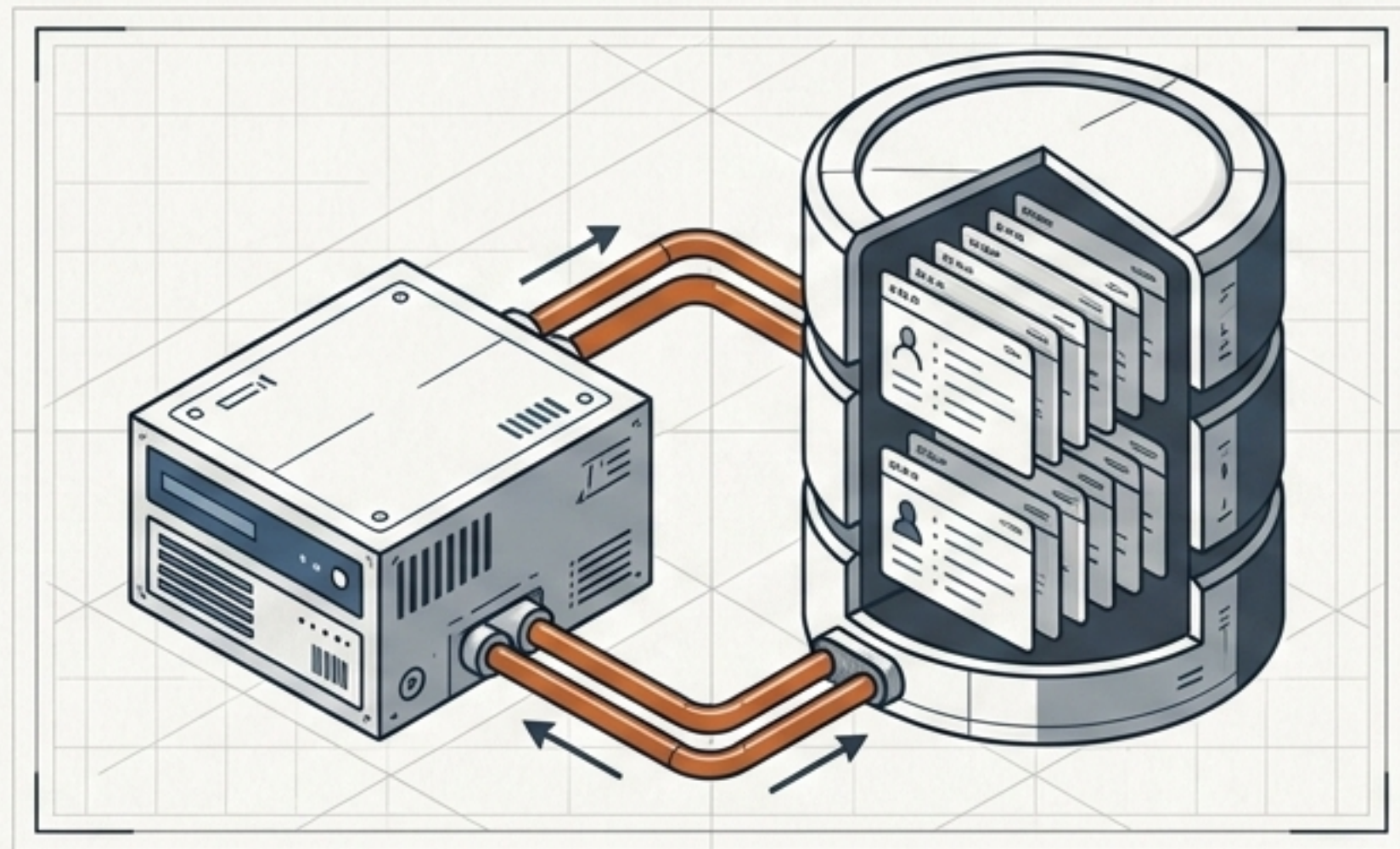
API

식당의 웨이터. 앱이 서버에 특정
데이터를 요청(호출)하는 창구.
고유한 URL과 함께 GET, POST,
PUT, DELETE 방식 사용.

JSON

응답의 표준 규격. 서버가 API 요청을
받고 클라이언트에게 데이터를
돌려줄 때 사용하는 텍스트 포맷.
{위치: 서울, 날씨: 맑음}

영구적인 기억 장치: 데이터베이스(DB)와 SQL



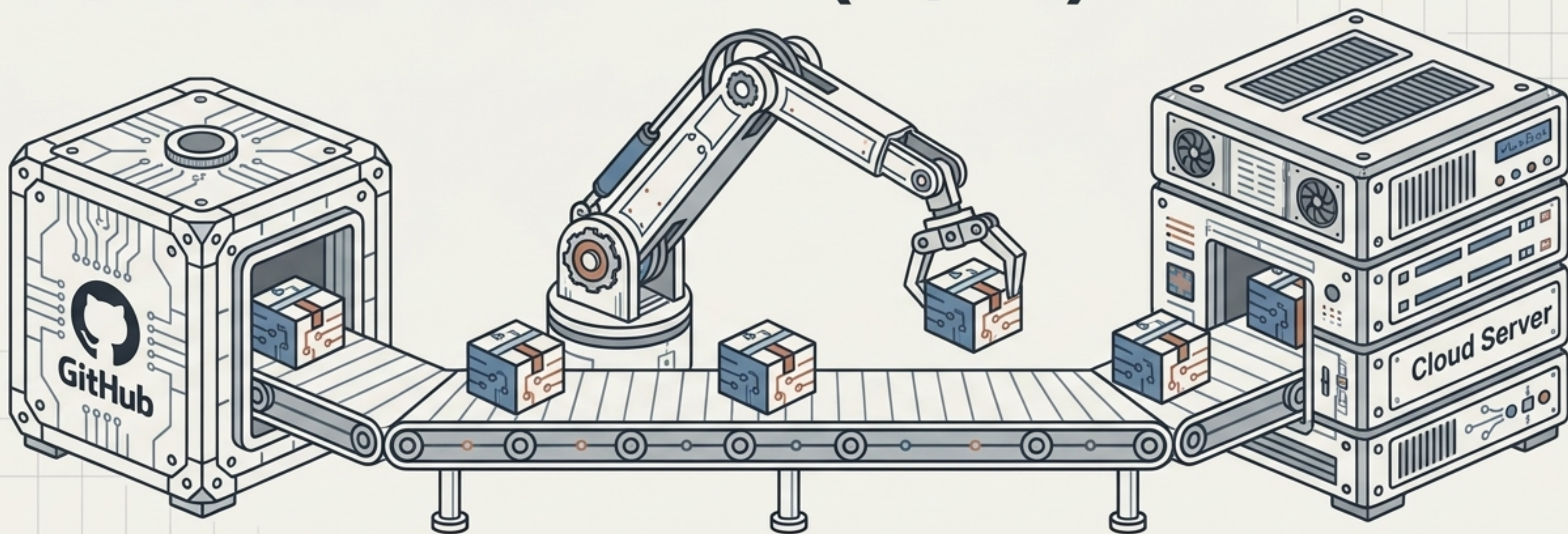
데이터베이스 (DB)

정보의 금고. 회원 정보, 게시판 글 등 앱 구동에 필요한 모든 실제 데이터가 영구적으로 저장되는 곳. (MySQL, MongoDB).

SQL

금고 전용 명령어. 백엔드 서버가 이 데이터베이스에서 특정 데이터를 꺼내오거나, 새로운 데이터를 저장할 때 사용하는 전용 통신 언어.

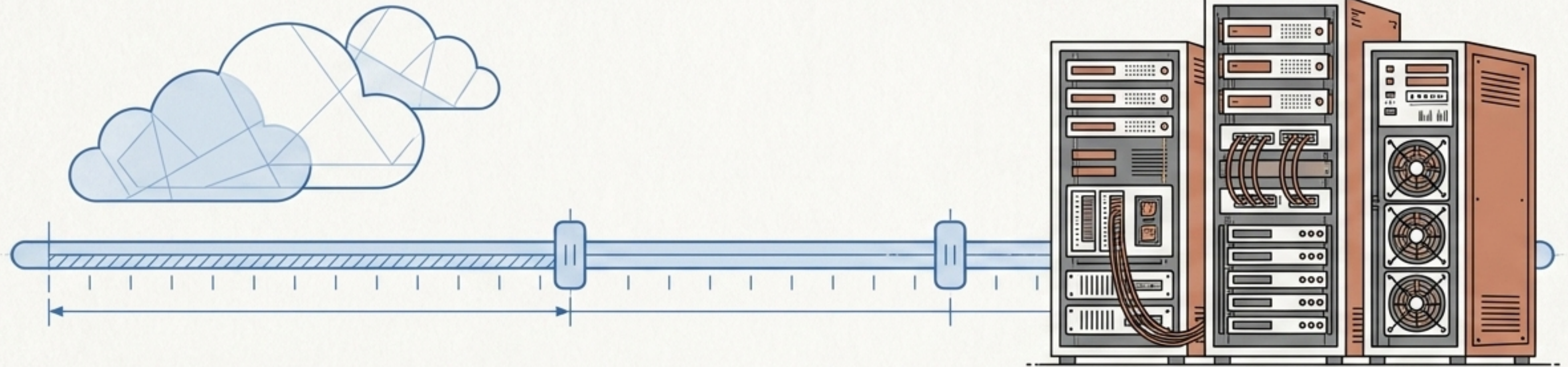
자동화된 배포 파이프라인 (CI/CD)



코드를 수정할 때마다 수동으로 서버에 접속해 올릴 필요가 없습니다. 실무에서는 이를 완벽히 자동화합니다.

CI/CD (지속적 통합/배포): 깃허브(GitHub)에 새로운 소스를 업데이트(Push)하기만 하면, 시스템이 이를 감지하여 자동으로 빌드하고 클라우드 서버에 반영(배포)하는 마법 같은 자동화 시스템.

배포(Deployment): 내 컴퓨터에서 세상 밖으로



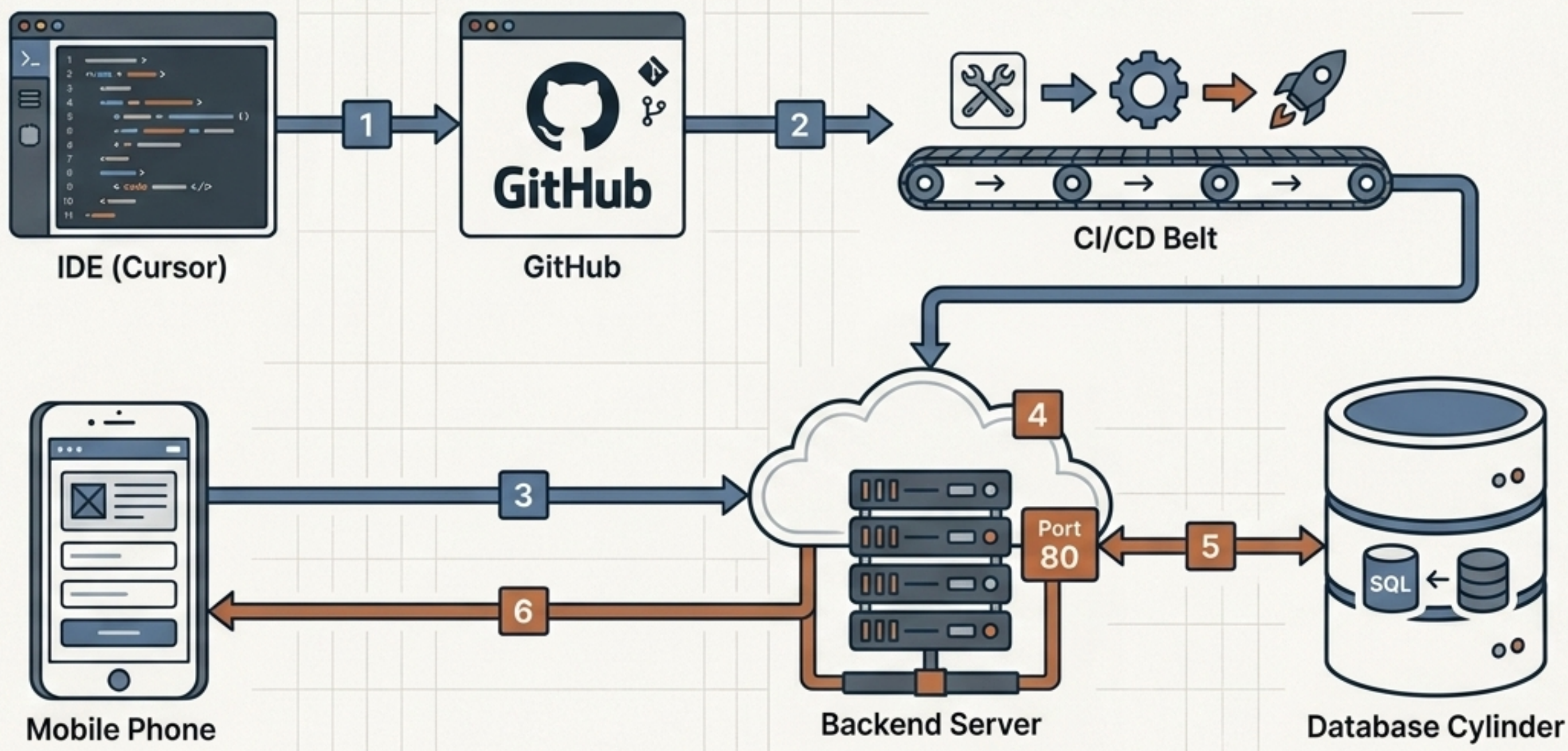
서비스형 배포 (초보자 & 빠른 런칭)

도구: Cloudflare Pages, Vercel
특징: 깃허브 프로젝트를 연결하거나 폴더를 드래그 앤 드롭만 하면 즉시 서버에 올려지고 접속 가능한 URL을 발급해 줍니다.

인프라형 배포 (전문가 & 완전한 제어)

도구: AWS, Azure
특징: 실제 데이터센터에 나만의 서버(컴퓨터)를 할당받는 방식. **리눅스(Linux)** 환경을 직접 다뤄야 하므로 복잡하지만 세밀한 제어가 가능합니다.

바이브 코딩 풀스택 아키텍처 (최종 설계도)



1. IDE (Cursor)에서 AI 프롬프트로 소스 코드 작성.	2. 코드를 GitHub에 저장, CI/CD를 통해 클라우드로 자동 배포.	3. 사용자가 스마트폰에서 HTTP를 통해 GET API 호출.	4. 80번 포트로 떠 있는 백엔드 서버가 요청 수신.	5. 백엔드가 SQL을 사용해 데이터베이스에서 정보 조회.	6. JSON 형태로 프론트에 반환하여 화면에 출력.
--------------------------------------	---	-------------------------------------	--------------------------------	----------------------------------	-------------------------------

바이브 코더의 트러블슈팅 체크리스트

[✓] 환경 세팅 [IDE & LLM]

내 공장(VS Code/Cursor)에 AI
모듈이 정상적으로 연결되어 맥락을
잘 파악하고 있는가?

[✓] 소스 관리 [Git/GitHub]

AI가 짜준 코드가 로컬 컴퓨터에만
머물지 않고 원격 저장소에 안전하게
Push 되었는가?

[✓] 통신 확인 [Port & API]

백엔드 서버가 정확한 포트(예 : 80,
3306)로 로컬호스트에 떠 있으며,
프론트가 올바른 API URL로
호출하고 있는가?

[✓] 배포 상태 [CI/CD & Cloud]

배포 파이프라인이 정상 작동하여
실제 사용자 URL로 연결되는가?



아키텍처의 맥락이 뚫리면, 코딩은 AI가 알아서 합니다.

이제 진짜 ‘시스템 설계’를 시작해 보세요.